SIDE PROJECT · ALGORITHMIC TRADING

```
if mean >= Decimal(0.33):
    place_buy_order()
    watch_order_book()
    place_sell_order()
```

Order Book Imbalance Bot

A Python bot that reads the buy/sell imbalance on Binance's order book in real time and tries to get ahead of the price.

Python | WebSocket | Binance API | Async/Await | Order Book

THE IDEA

Before it shows up on the chart, the price has already moved inside the order book.

When buy volume (bid) clearly outweighs sell volume (ask) right at the top of the book, there's more pressure to buy than to sell: the price tends to rise, and vice versa. The bot tracks this imbalance in real time over a 10-second window and, if the signal is strong enough and the trade would be profitable, opens a position on its own.

01

Real-time data

A Binance WebSocket listening to bookTicker and the top 10 rows of the book (depth10).

02

An imbalance metric

A moving average of (bid − ask) / (bid + ask): a number between −1 and 1 that tells you who's winning.

03

Automatic execution

Opening, monitoring and closing the order are all handled by the bot, with no manual step.

ARCHITECTURE

From the live data stream to closing the position



GETTING THE NUMBERS RIGHT

Why it's built this way

Decimal instead of float

Prices can't tolerate rounding errors: Python's decimal module guarantees exact precision on every calculation, from price to estimated profit.

Quantizing to tickSize and minQty

Every price and quantity is rounded to Binance's actual book constraints, so an order is never rejected over one extra decimal.

User Data Stream

A dedicated listenKey reports in real time when an order gets filled, with no need to poll the API.

Fallback to a market order

If the book moves and the closing limit order doesn't fill in time, the bot still closes the position with a market order.

SOURCE CODE

binance_orderbook_bot.py

binance_orderbook_bot.py

```python
import asyncio
import websockets
import requests
import json
from decimal import *
import decimal
from binance import *
from binance.enums import *
import statistics
import time
import pandas as pd

getcontext().prec = 9
money = 30
symbol = "OCEANBUSD"

api_key = "YOUR_BINANCE_API_KEY"
api_secret = "YOUR_BINANCE_API_SECRET"

response = requests.post('https://api.binance.com/api/v3/userDataStream', headers={'X-MBX-APIKEY': api_key})
boh = response.json()
listenKey = boh["listenKey"]

client = Client(api_key, api_secret)


async def main():
```

SOURCE CODE

binance_orderbook_bot.py

binance_orderbook_bot.py

```
28      async with websockets.connect(f"wss://stream.binance.com:9443/stream?streams={listenKey}/{symbol.lower()}@bookTicker/{symbol.lower()}@depth10@100ms
        → ") as websocket:
29          v_imb_list = []
30          nexttime = time.time()
31          info = client.get_symbol_info(symbol=symbol)
32          tickp = float(info["filters"][0]["tickSize"])
33          tickq = float(info["filters"][2]["minQty"])
34          while True:
35              data = json.loads(await websocket.recv())
36              if data["stream"] != f"{symbol.lower()}@bookTicker":
37                  continue
38              else:
39                  bid = Decimal(data["data"]["b"])
40                  ask = Decimal(data["data"]["a"])
41                  v_bid = decimal.Decimal(data["data"]["B"])
42                  v_ask = decimal.Decimal(data["data"]["A"])
43
44                  if tickq == 1:
45                      buyqt = int(money/bid)
46                  else:
47                      buyqt = Decimal(money / bid).quantize(Decimal(f"{tickq}"), rounding=ROUND_DOWN)
48
49                  profit = -money + (buyqt * ask) - Decimal(money * 0.075 / 100 * 2)
50
51                  v_imb = Decimal((v_bid - v_ask) / (v_bid + v_ask)).quantize(Decimal(".01"), rounding=ROUND_DOWN)
52                  v_imb_list.append(v_imb)
53                  mean = Decimal(statistics.mean(v_imb_list)).quantize(Decimal(".01"), rounding=ROUND_DOWN)
```

SOURCE CODE

binance_orderbook_bot.py

binance_orderbook_bot.py

```python
                time_passed = int(time.time() - nexttime)

                if time_passed > 0 and time_passed % 10 == 0:
                    v_imb_list.clear()
                    if profit <= 0:
                        continue
                    else:
                        bid = decimal.Decimal(bid) + decimal.Decimal(tickp)
                        ask = decimal.Decimal(ask) - decimal.Decimal(tickp)
                        bid1 = float(bid)
                        ask1 = float(ask)
                        bid = Decimal(bid).quantize(Decimal(f"{tickp}"), rounding=ROUND_DOWN)
                        ask = Decimal(ask).quantize(Decimal(f"{tickp}"), rounding=ROUND_DOWN)

                        if Decimal(-1) <= mean <= Decimal(0.32):
                            continue
                        elif Decimal(0.33) <= mean <= Decimal(1):
                            client.create_order(
                                symbol=symbol,
                                side=SIDE_BUY,
                                type=ORDER_TYPE_LIMIT,
                                timeInForce=TIME_IN_FORCE_GTC,
                                quantity=buyqt,
                                price=bid)

                            flag = False
                            count = 0
```

SOURCE CODE

binance_orderbook_bot.py

binance_orderbook_bot.py

```
81                          count3 = 0
82
83                          while True:
84                              data = json.loads(await websocket.recv())
85                              if data["stream"] != str(listenKey) and count3 == 0:
86                                  if data["stream"] == f"{symbol.lower()}@depth10@100ms" and count != 0:
87                                      dataa = pd.DataFrame(data["data"]["asks"]).head(6)
88                                      dataa.columns = ["price", "qt"]
89                                      dataa["price"] = pd.to_numeric(dataa["price"])
90                                      data = pd.DataFrame(data["data"]["bids"]).head(6)
91                                      data.columns = ["price", "qt"]
92                                      data["price"] = pd.to_numeric(data["price"])
93                                      data["price"] = data["price"] - bid1
94                                      print(data)
95                                      print(dataa)
96                                      if 0 in data["price"].values and ask1 <= dataa["price"][5]:
97                                          continue
98                                      else:
99                                          open_order = client.get_open_orders(symbol=symbol)
100                                         try:
101                                             print(open_order)
102                                             orderid = open_order[0]["orderId"]
103                                         except IndexError:
104                                             count3 += 1
105                                             continue
106                                         client.cancel_order(
107                                             symbol=symbol,
```

SOURCE CODE

binance_orderbook_bot.py

binance_orderbook_bot.py

```
                                orderId=orderid)

                        flag = True
                        break
                else:
                    continue
            elif data["data"]["e"] != "executionReport" and count3 == 0:
                continue
            elif data["data"]["X"] != "FILLED" and count3 == 0:

                while True:
                    data = json.loads(await websocket.recv())
                    if data["stream"] == f"{symbol.lower()}@depth10@100ms":
                        dataa = pd.DataFrame(data["data"]["asks"]).head(6)
                        dataa.columns = ["price", "qt"]
                        dataa["price"] = pd.to_numeric(dataa["price"])
                        data = pd.DataFrame(data["data"]["bids"]).head(6)
                        data.columns = ["price", "qt"]
                        data["price"] = pd.to_numeric(data["price"])
                        data["price"] = data["price"] - bid1
                        print(data)
                        print(dataa)
                        if 0 in data["price"].values and ask1 <= dataa["price"][5]:
                            count += 1
                            break
                        else:
                            open_order = client.get_open_orders(symbol=symbol)
```

SOURCE CODE

binance_orderbook_bot.py

binance_orderbook_bot.py

```
                                try:
                                    print(open_order)
                                    orderid = open_order[0]["orderId"]
                                except IndexError:
                                    count3 += 1
                                    break
                                client.cancel_order(
                                    symbol=symbol,
                                    orderId=orderid)
                                flag = True
                                break
                            else:
                                continue

                    else:
                        client.create_order(
                            symbol=symbol,
                            side=SIDE_SELL,
                            type=ORDER_TYPE_LIMIT,
                            timeInForce=TIME_IN_FORCE_GTC,
                            quantity=buyqt,
                            price=ask)

                        count2 = 0

                        while True:
                            data = json.loads(await websocket.recv())
```

SOURCE CODE

binance_orderbook_bot.py

binance_orderbook_bot.py

```
162                 if data["stream"] != str(listenKey):
163                     if data["stream"] == f"{symbol.lower()}@depth10@100ms" and count2 != 0:
164                         data = pd.DataFrame(data["data"]["asks"]).head(6)
165                         data.columns = ["price", "qt"]
166                         data["price"] = pd.to_numeric(data["price"])
167                         data["price"] = data["price"] - ask1
168                         if 0 in data["price"].values:
169                             continue
170                         else:
171                             open_order = client.get_open_orders(symbol=symbol)
172                             try:
173                                 orderid = open_order[0]["orderId"]
174                             except IndexError:
175                                 flag = True
176                                 break
177                             client.cancel_order(
178                                 symbol=symbol,
179                                 orderId=orderid)
180
181                             client.order_market_sell(
182                                 symbol=symbol,
183                                 quantity=buyqt)
184
185                             flag = True
186                             break
187                     else:
188                         continue
```

SOURCE CODE

binance_orderbook_bot.py

binance_orderbook_bot.py

```
189
190                 elif data["data"]["e"] != "executionReport":
191                     continue
192                 elif data["data"]["X"] != "FILLED":
193                     orderid = data["data"]["i"]
194
195                     while True:
196                         data = json.loads(await websocket.recv())
197                         if data["stream"] == f"{symbol.lower()}@depth10@100ms":
198                             data = pd.DataFrame(data["data"]["asks"]).head(6)
199                             data.columns = ["price", "qt"]
200                             data["price"] = pd.to_numeric(data["price"])
201                             data["price"] = data["price"] - ask1
202                             if 0 in data["price"].values:
203                                 count2 += 1
204                                 break
205                             else:
206                                 open_order = client.get_open_orders(symbol=symbol)
207                                 try:
208                                     orderid = open_order[0]["orderId"]
209                                 except IndexError:
210                                     flag = True
211                                     break
212                                 client.cancel_order(
213                                     symbol=symbol,
214                                     orderId=orderid)
215
```

SOURCE CODE

binance_orderbook_bot.py

binance_orderbook_bot.py

```
                                                client.order_market_sell(
                                                    symbol=symbol,
                                                    quantity=buyqt)
                                                flag = True
                                                break
                                            else:
                                                continue
                                        else:
                                            flag = True
                                            break
                                        if flag:
                                            break
                                    if flag:
                                        break
                                if flag:
                                    break
                            if flag:
                                break
                        else:
                            continue

asyncio.run(main())
```

RISKS AND NEXT STEPS

What's missing, honestly

- No automatic reconnection if the WebSocket drops
- No explicit stop-loss or dynamic position sizing
- One symbol at a time, no historical backtesting
- Next steps: network error handling, structured logging, multi-symbol support

DISCLAIMER

A learning project for exploring order books, WebSockets and asynchronous programming.
This is not financial advice and it isn't built for real capital: trading carries a real
risk of losing the money you put into it.